

# A bit of theory as far as relevant to the implementation of `class LagrangeMesh`

The necessary theory - for non-reduced axes - is found in [Ryssens et al, PHYSICAL REVIEW C 92, 064318 \(2015\)](#) section III.B Lagrange-mesh representation.

## Non-reduced axes

(As from [Ryssens et al, PHYSICAL REVIEW C 92, 064318 \(2015\)](#) section III.B Lagrange-mesh representation)

## Grid points

Using  $M$  evenly space grid point at distance  $dx$ , the boundaries of the box are  $[-\frac{M}{2}dx, \frac{M}{2}dx]$ . The width of the box is  $L = Mdx$ . We require  $M = 2N$  to be even such that there are  $N = \frac{M}{2}$  grid points on each side of the origin and the origin is avoided as a grid point.

For non-reduced axes (where the shift parameter  $\sigma$  is required be zero) the grid points are:  
[eq 1]

$$\pm\frac{1}{2}dx, \pm\frac{3}{2}dx, \dots, \pm\frac{N-3}{2}dx, \pm\frac{N-1}{2}dx$$

or

[eq 2]

$$x_{\pm i} = \pm(\frac{1}{2} + i)dx$$
$$i = 0..N-1$$

in order:

[eq 3]

$$-\frac{2N-1}{2}dx, -\frac{2N-3}{2}dx, \dots, -\frac{3}{2}dx, -\frac{1}{2}dx, \frac{1}{2}dx, \frac{3}{2}dx, \dots, \frac{2N-3}{2}dx, \frac{2N-1}{2}dx$$

or

[eq 4]

$$x_i = -\frac{2N-1-2i}{2} = \frac{1}{2} + i - N, i = 0..2N-1$$

## Basis functions

The basis functions are plane waves:

[eq 5]

$$\phi_k(x) = \frac{1}{\sqrt{L}} \exp\left(\frac{2\pi i}{L} kx\right)$$

$$k = \pm \frac{1}{2}, \pm \frac{3}{2}, \dots, \pm \frac{N-1}{2},$$

(Note that  $i$  is the imaginary unit here).

## Interpolation

The Lagrange interpolation functions are:

[eq 6]

$$f_i(x) = \frac{1}{2N} \frac{\sin\left(\frac{\pi}{dx}(x - x_i dx)\right)}{\sin\left(\frac{\pi}{dx} \frac{x - x_i dx}{2N}\right)}$$

By construction, the Lagrange interpolation functions  $f_i(x)$  have the property of being equal to 1 at the  $i$ -th mesh point,  $x_i$ , and 0 at all others:

[eq 7]

$$f_i(x_j) = \delta_{ij}$$

Note that to evaluate  $f_i(x_i)$ , both the numerator and the denominator are 0, and we need to invoke l'Hôpital's rule

[eq 7.1]

$$\begin{aligned} & \lim_{x \rightarrow x_i} \frac{1}{2N} \frac{\sin\left(\frac{\pi}{dx}(x - x_i dx)\right)}{\sin\left(\frac{\pi}{dx} \frac{x - x_i dx}{2N}\right)} \\ &= \frac{1}{2N} \lim_{x \rightarrow x_i} \frac{\sin'\left(\frac{\pi}{dx}(x - x_i dx)\right)}{\sin'\left(\frac{\pi}{dx} \frac{x - x_i dx}{2N}\right)} \\ &= \frac{1}{2N} \lim_{x \rightarrow x_i} \frac{\frac{\pi}{dx} \cos\left(\frac{\pi}{dx}(x - x_i dx)\right)}{\frac{\pi}{2N dx} \cos\left(\frac{\pi}{dx} \frac{x - x_i dx}{2N}\right)} \\ &= \frac{2N}{2N} \lim_{x \rightarrow x_i} \frac{\cos(0)}{\cos(0)} = 1 \end{aligned}$$

The arguments of the two sine functions are the same, apart from a factor  $1/2N$ :

[eq 8]

$$f_i(x) = \frac{1}{2N} \frac{\sin(A_i(x))}{\sin\left(\frac{A_i(x)}{2N}\right)}$$

$$A_i(x) = \frac{\pi}{dx}(x - x_i dx) = \pi\left(\frac{x}{dx} - x_i\right)$$

This can be exploited for efficiency in a computation.

An arbitrary function  $h(x)$  taking the values  $h_i = h(x_i)$  on the grid points can be interpolated on an arbitrary point  $x \in [-Ndx, Ndx]$  as:

[ea 9]

$$h(x) = \sum_{i=0}^{2N-1} f_i h(x_i)(x)$$

This is essentially a dot product  $\mathbf{f} \cdot \mathbf{h}$ .

## Derivatives

The 1st and 2nd order derivatives of the Lagrange interpolation functions are:

[eq 10]

$$D_{ji}^{(1)} = \left. \frac{df_i(x)}{dx} \right|_{x=x_j} = \begin{cases} (-1)^{i-j} \frac{\pi}{2Ndx} \frac{1}{\sin(\pi(i-j)/2N)}, & \text{for } i \neq j. \\ 0, & \text{for } i = j. \end{cases}$$

$$D_{ji}^{(2)} = \left. \frac{d^2 f_i(x)}{dx^2} \right|_{x=x_j} = \begin{cases} (-1)^{i-j+1} 2 \left( \frac{\pi}{2Ndx} \right)^2 \frac{\cos(\pi(i-j)/2N)}{\sin^2(\pi(i-j)/2N)}, & \text{for } i \neq j. \\ -\frac{\pi^2}{3dx^2} \left( 1 - \frac{1}{(2N)^2} \right), & \text{for } i = j. \end{cases}$$

By applying these to the expansion  $\phi(x) = \sum_{i=0}^{2N-1} \phi(x_i) f_i(x)$  we obtain

[eq 11]

$$\frac{d\mathbf{h}}{dx} = \left. \frac{dh(x)}{dx} \right|_{x=x_j} = \sum_{i=0}^{2N-1} \left. \frac{df_i(x)}{dx} \right|_{x=x_j} h(x_i) = \sum_{i=0}^{2N-1} D_{ji}^{(1)} h(x_i) = \mathbf{D}^{(1)} \mathbf{h}$$

So, the column vector  $\frac{d\mathbf{h}}{dx}$  of the derivatives of  $\mathbf{h}$  at all grid points is found as a matrix product of  $\mathbf{D}^{(1)}$  with the column vector of the values of  $\mathbf{h}$  at all the gridpoints. As the 2nd derivative forms a matrix as well, we also have

[eq 12]

$$\frac{d^2 \mathbf{h}}{dx^2} = \mathbf{D}^{(2)} \mathbf{h}$$

## Reduced axes

### Grid points

On reduced axes only the  $N$  grid points to the right of the origin are kept:

[eq 13]

$$\frac{1}{2}dx, \frac{3}{2}dx, \dots, \frac{2N-3}{2}dx, \frac{2N-1}{2}dx$$

or

[eq 14]

$$x_i = \left(i + \frac{1}{2}\right)dx$$

$$i = 0..N-1$$

(In the case of reduced axis a shift  $\sigma < dx$  may be subtracted from each grid point.)

## Interpolation

An arbitrary function  $h(x)$ ,  $x \in [0, Ndx]$  taking the values  $h_i = h(x_i)$  on the grid points on a reduced axis can be interpolated as :

[eq 15]

$$h(x) = \sum_{i=0}^{2N-1} h_i [f_i(x) \pm f_{-i}(x)]$$

where  $+$ , resp.  $-$ , is selected if  $h(x)$  is symmetric, resp. antisymmetric, and  $f_{-i}(x)$  is the Lagrange interpolation function corresponding to the  $i$ -th grid point to the left of the origin:  
[eq 16]

$$f_{-i}(x) = \frac{1}{2N} \frac{\sin(\frac{\pi}{dx}(x - x_{-i}dx))}{\sin(\frac{\pi}{dx} \frac{x - x_{-i}dx}{2N})} = \frac{1}{2N} \frac{\sin(\frac{\pi}{dx}(x + (\frac{1}{2} + i)dx))}{\sin(\frac{\pi}{dx} \frac{x + (\frac{1}{2} + i)dx}{2N})}$$

This time we have a sum or a difference of two dot products  $\mathbf{h} \cdot \mathbf{f}_+ \pm \mathbf{h} \cdot \mathbf{f}_-$

Again the arguments of the two sine functions are the same, apart from a factor  $1/2N$ .

## Derivatives

The formula for the derivative of a function  $h$  expanded on a reduced grid is found easily by extending the column vector  $\mathbf{h}$  (of length  $N$ ) on the reduced grid as

[eq 17]

$$\begin{bmatrix} \pm \mathbf{g} \\ - \\ \mathbf{h} \end{bmatrix}$$

with  $\mathbf{g}$  containing the same elements as  $\mathbf{h}$  but in reverse order, if the function  $h$  is symmetric, and its negatives in reverse order, if it antisymmetric as if they were mirrored by a horizontal line between the two, and then forming the matrix product

[eq 18]

$$\mathbf{D} \begin{bmatrix} \pm \mathbf{g} \\ \mathbf{h} \end{bmatrix} = \begin{bmatrix} \mathbf{D}^- \\ \mathbf{D}^+ \end{bmatrix} \begin{bmatrix} \pm \mathbf{g} \\ \mathbf{h} \end{bmatrix} = \begin{bmatrix} \mathbf{D}^- \begin{bmatrix} \pm \mathbf{g} \\ \mathbf{h} \end{bmatrix} \\ - \\ \mathbf{D}^+ \begin{bmatrix} \pm \mathbf{g} \\ \mathbf{h} \end{bmatrix} \end{bmatrix}$$

The upper half of  $\mathbf{D}$ ,  $\mathbf{D}^-$ , produces the derivatives on the negative  $x$ -axis, and the lower half of  $\mathbf{D}$ ,  $\mathbf{D}^+$ , produces the derivatives on the positive  $x$ -axis. The upper part and the lower part are obviously related by symmetry: if the function  $h(x)$  is symmetric, then the derivative is antisymmetric and *vice versa*. Thus we are only interested in the lower part. Writing  $\mathbf{D}^+$  as  $[\mathbf{D}_{ll} | \mathbf{D}_{lr}]$  ( $ll$  stands for lower-left and  $lr$  for lower-right quadrant of  $\mathbf{D}$ ) we find:

[eq 19]

$$\mathbf{D}^+ \begin{bmatrix} \pm \mathbf{g} \\ \mathbf{h} \end{bmatrix} = [\mathbf{D}_{ll} | \mathbf{D}_{lr}] \begin{bmatrix} \pm \mathbf{g} \\ \mathbf{h} \end{bmatrix} = [\mathbf{E}_{ll} | \mathbf{D}_{lr}] \begin{bmatrix} \pm \mathbf{h} \\ \mathbf{h} \end{bmatrix}$$

where  $\mathbf{E}_{ll}$  is obtained from  $\mathbf{D}_{ll}$  by reversing the order of the rows. Reversing the order of the rows of  $\mathbf{D}_{ll}$  forces us to also reverse the order of the elements of  $\pm \mathbf{g}$ , which then becomes, by definition  $\pm \mathbf{h}$ . Finally, we obtain:

[eq 20]

$$\pm \mathbf{E}_{ll} \mathbf{h} + \mathbf{D}_{lr} \mathbf{h} = [\mathbf{D}_{lr} \pm \mathbf{E}_{ll}] \mathbf{h}$$

or

[eq 21]

$$\frac{d\mathbf{h}}{d\mathbf{x}} = [\mathbf{D}_{lr}^{(1)} \pm \mathbf{E}_{ll}^{(1)}] \mathbf{h}$$

where we have reintroduced the superscript <sup>(1)</sup> to indicate the first order derivative. By the same reasoning, we obtain for the 2nd derivative:

[eq 22]

$$\frac{d^2\mathbf{h}}{d\mathbf{x}^2} = [\mathbf{D}_{lr}^{(2)} \pm \mathbf{E}_{ll}^{(2)}] \mathbf{h}$$

## ***N*-dimensional grids**

The full Cartesian 3D representation of a function  $h(\mathbf{r})$ , where  $\mathbf{r} = [x \ y \ z]$  (3D case), is then provided (for the 3D case) by

[eq 23]

$$h(\mathbf{r}) = \sum_{ijk} h_{ijk} f_i(x) f_j(y) f_k(z)$$

where the number of discretization points does not have to be the same in each direction.

Note that  $f_i$ ,  $f_j$  and  $f_k$  are generally different objects, even if accidentally the indices  $i$ ,  $j$  and  $k$  are identical, as they pertain, resp., to the  $x$ -axis, the  $y$ -axis and the  $z$ -axis.

## **Derivatives**

In this case, the derivative matrices  $\mathbf{D}^{(1)}$  and  $\mathbf{D}^{(2)}$  have to be set up separately for each direction, taking into account whether the axis is reduced or not.

## **Basis functions**

The basis functions are plane wave products of the different axes:

[eq 24]

$$\begin{aligned} \Phi_{klm}(x, y, z) &= \phi_k(x) \phi_l(y) \phi_m(z) = \frac{1}{\sqrt{L_x}} \frac{1}{\sqrt{L_y}} \frac{1}{\sqrt{L_z}} \exp\left(\frac{2\pi i}{L_x} kx\right) \exp\left(\frac{2\pi i}{L_y} ly\right) \exp\left(\frac{2\pi i}{L_z} mz\right) \\ &= \frac{1}{\sqrt{L_x L_y L_z}} \exp\left(2\pi i \left(\frac{kx}{L_x} + \frac{ly}{L_y} + \frac{mz}{L_z}\right)\right) \end{aligned}$$

$$= \frac{1}{\sqrt{L_x L_y L_z}} \exp(2\pi i(\mathbf{k} \cdot \mathbf{r}))$$

$$\mathbf{k} = \begin{bmatrix} \frac{k}{L_x} & \frac{l}{L_y} & \frac{m}{L_z} \end{bmatrix}$$

$$\mathbf{x} = \begin{bmatrix} x & y & z \end{bmatrix}$$

Note that  $\phi_k$ ,  $\phi_l$  and  $\phi_m$  are generally different objects, even if accidentally the indices  $k$ ,  $l$  and  $m$  are identical, as they pertain, resp., to the  $x$ -axis, the  $y$ -axis and the  $z$ -axis.

## Interpolation

As described by eq 23 Interpolating a scalar quantity  $h$  at a single point requires a sum over all grid points which may be costly (speaking of working interactively). If  $h$  needs to be interpolated on a large number of points,  $p$ ,

$$\mathbf{r} = \begin{bmatrix} x_0 & y_0 & z_0 \\ x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ \vdots & \vdots & \vdots \\ x_{p-1} & y_{p-1} & z_{p-1} \end{bmatrix}$$

it will be advantageous to move the loop over the points  $0..p-1$  inside the loop over  $ijk$  product  $h_{ijk}f_i(x)f_j(y)f_k(z)$ .

In case  $h$  is not a scalar quantity but a tensor it is advantageous to move the loop over the components between the loop over the  $p$  interpolation points and the loop over  $ijk$ :

```
for all grid points ijk:
    for all components h of H
        for all interpolation points 0..p
            accumulate h_ijk * f_i(x_p) * f_j(y_p) * f_k(z_p) over ijk
```

The inner loop can be evaluated as a function over a numpy array:

```
for all grid points ijk:
    for all components h of H
        h(r) += h_ijk * f_i(r[:,0]) * f_j(r[:,1]) * f_k(r[:,2])
```

Furthermore, in case e.g. the  $x$  axis is reduced, according to eq 15 one must replace  $f_i(x)$  by  $(f_i(x) \pm f_{-i}(x))$ .